Lesson 10 — Exam Preparation

INTE 22253 Distributed Systems and Cloud Computing | Final exam: 02.11.2026, 2 hours, closed book, 50% of the module | Built from Lessons 1–9 (Lectures 01–08, Lab 01, Assignment 01)

HOW TO USE THIS LESSON

1. **Part A**: learn the definitions word for word; most answers start with one.
2. **Part B**: for each lesson, the questions most likely to be asked, with the points a full answer must contain. Practise writing them from memory.
3. **Part C**: the comparison tables examiners love. Be able to draw each one.
4. **Part D**: a full mock paper with model answers (also in the Practice questions tab).

ANSWER TECHNIQUE FOR A 2-HOUR CLOSED-BOOK PAPER

Define → explain → example → diagram. Start every answer with the definition; give one point per mark; add a concrete example (bank, Google, AWS, online shop); draw a labelled diagram whenever the topic has one (architecture, 2PC, CAP triangle, hypervisor stack, GFS, HA design). For "compare" questions always use a **table** with 4–6 rows. For "discuss trade-offs" give **both** sides and say when each is preferred. Budget about 1.2 minutes per mark.

Part A — Definitions to memorise

Lesson 1 — Distributed systems	
Distributed system	A collection of independent computers that appears to its users as a single coherent system, communicating over a network to achieve a common goal.
Transparency	Hiding the distribution from users (access, location, migration, replication, concurrency, failure).
Scalability	Ability to grow capacity, preferably by adding nodes (horizontal scaling), while performing well.
Partial failure	Some components fail while others keep running.
CAP theorem	A distributed data store can guarantee at most two of Consistency, Availability and Partition tolerance; under a partition choose C or A.
Middleware	Software layer that hides heterogeneity and gives a uniform interface.
Openness / heterogeneity	Built on standard published interfaces / nodes differ in hardware, OS, network, language.
Lesson 2 — Cloud computing	
Cloud computing	Delivery of computing as a service rather than a product, where shared resources, software and information are provided as a metered service over a network: on-demand IT, pay-as-you-go.
NIST characteristics	On-demand self-service, broad network access, resource pooling, rapid elasticity, measured service.
Big data	Large pools of data that can be captured, communicated, aggregated, stored and analysed to deliver insights and value (5 V's).
Multi-tenancy	Multiple customers share the same physical infrastructure securely and independently.
IaaS / PaaS / SaaS	Virtual infrastructure (you manage OS and up) / platform to build and deploy apps (you manage app and data) / complete software over the Internet (you just use it).
Public / private / hybrid / multi-cloud	Third-party shared / one organisation only / public + private with portability / several public providers.
Serverless (FaaS)	Developers don't manage servers; functions run on events, scale to zero, pay per invocation.
Edge / fog computing	Processing close to where data is generated (<10 ms) / a middle layer between edge and cloud that aggregates edge data (10–50 ms).
Shared responsibility	Provider secures OF the cloud (physical, network hardware, hypervisor); customer secures IN the cloud (data, IAM, network config, apps).

Zero Trust	"Never trust, always verify": verify explicitly, least privilege, assume breach.
FinOps	Bringing financial accountability to cloud spending.
Lesson 3 — NFV, SDN, Docker, HA	
NFV	Decoupling network functions (firewall, encryption) from dedicated hardware and running them on virtual servers.
SDN	Separating the network control plane from the data plane and centralising control in software.
Container / image	Running instance / read-only template built from a Dockerfile in layers.
Availability zone	Cluster of data centres in a region with independent power, cooling and redundant networking.
RTO / RPO	Maximum acceptable downtime / maximum acceptable data loss measured in time.
Lesson 4 — Consistency, fault tolerance	
Consistency model	A contract between the data store and processes specifying what values a read may return after writes.
Strict / sequential / causal / eventual	Latest write in real time / one interleaving for all / same order only for causally related writes / converge when updates stop.
Fault → error → failure	Defect in a component → incorrect internal state → visible service breakdown.
Fault tolerance	Continuing to operate correctly despite failures of some components.
Byzantine failure	Arbitrary output at arbitrary times, including malicious behaviour.
Atomic multicast	A message is delivered to every non-faulty group member or to none.
2PC	Coordinator collects YES/NO votes; all YES ⇒ COMMIT, any NO ⇒ ABORT; blocking if the coordinator fails after YES votes.
PACELC	If Partition: A or C; Else: Latency or Consistency.
Lesson 5 — Infrastructure and virtualization	
Virtualization	Creating a software-based version of a resource so one physical machine behaves like several.
Hypervisor (VMM)	Software that creates, runs and manages VMs, allocating hardware and isolating them.
Type 1 / Type 2	Runs directly on hardware (ESXi, Hyper-V, Xen, KVM) / runs as an app on a host OS (VirtualBox, Workstation).
Cloud infrastructure	Hardware and software (servers, storage, networking, virtualization) needed to support cloud computing.
Resource pooling	Physical compute, storage and network combined into a shared pool from which slices are allocated on demand.
Lesson 6 — Management, Kubernetes	
Cloud management	Controlling and monitoring cloud resources, storage, networks, apps, security policies and access.
Infrastructure as Code	Managing infrastructure through code instead of manual configuration.
Load balancer	Distributes incoming traffic among multiple servers or instances.
Auto-scaling	Automatically increasing or decreasing resources according to demand.
Kubernetes	Open-source container orchestration platform for deploying, managing and scaling containerized apps.
Pod	Smallest deployable unit: one or more containers sharing one IP and optional volumes, co-scheduled, ephemeral.
Resilience	Ability of a system to continue operating when failures occur.
Lesson 7 — Storage	

Block / file / object storage	Fixed-size addressable blocks / hierarchical files over NFS/SMB / immutable objects (data + metadata + key) over REST.
GFS / HDFS	Google's master/chunkserver DFS with 64 MB chunks / open-source NameNode/DataNode DFS with 128 MB blocks.
Durability	Data is not lost (S3: 11 nines); different from availability.
Erasure coding	Storing data plus parity so lost blocks can be rebuilt; ~1.5× overhead instead of 3×.
Lesson 8 — Server virtualization	
Full / para / hardware-assisted	Binary translation, unmodified guest / modified guest with hypercalls / CPU extensions (VT-x, AMD-V).
vMotion / Storage vMotion	Live migration of a running VM to another host / of its disk files to another datastore on the same host.
vSwitch	Software Ethernet switch inside ESX connecting VM vNICs to physical uplinks (vmnics).
DRS / HA / SRM	Auto load balancing across hosts / restart VMs after host failure / failover to a DR site.

Part B — Likely questions per lesson (with answer points)

Question	A full answer must contain
Lesson 1 — Introduction to distributed systems	
Define a distributed system; explain its characteristics.	Definition; no shared memory, message passing, single view, no global clock; resource sharing, scalability, concurrency, fault tolerance, no global clock, transparency (each with example).
Advantages and challenges of distributed systems.	Scalability, fault tolerance, performance, resource efficiency, geographic reach, cost; complexity, partial failures, consistency, synchronization, security, network reliability.
State and explain the CAP theorem with examples.	Three definitions; only two at once; partitions inevitable ⇒ C or A; CP bank vs AP social feed; triangle diagram.
Client-server vs P2P (vs microservices).	Structure, control, scalability, SPOF/churn, examples; diagram.
Explain how Google / AWS / banks use distributed systems.	Google: server farms, GFS chunks replicated, MapReduce. AWS: regions/AZs, EC2, S3. Banks: strong consistency, distributed transactions, HA.
Lesson 2 — Cloud computing	
Define cloud computing; NIST five characteristics.	Definition (service not product, metered, network); five characteristics with examples.
Compare IaaS, PaaS, SaaS (diagram).	Responsibility stack; what you manage; examples; best use.
Compare deployment models.	Public, private, hybrid (+ community, multi-cloud): definition, pros, cons, example.
Shared responsibility model; Zero Trust.	OF vs IN the cloud with S3 example; verify explicitly, least privilege, assume breach.
Serverless vs containers; edge vs fog vs cloud.	Not "no servers"; pros/cons; when to use; latency tiers and use cases (autonomous cars).
Why virtualization is the core enabler; FinOps; multi-cloud reasons.	Pooling, consolidation, isolation, rapid provisioning; right-size, reserved, spot, auto-scale, tiering, tags; lock-in, best-of-breed, compliance, cost, redundancy.
Lesson 3 — NFV, SDN, Docker, AWS HA, DR	
What is NFV? Advantages. NFV vs SDN.	Decouple functions from hardware onto VMs; six advantages; functions vs control-plane separation.
Docker architecture; image vs container; volumes.	Client → daemon ↔ registry; read-only template vs running instance; Dockerfile layers; ephemeral ⇒ volumes.

Design a highly available architecture on AWS.	VPC, 2 AZs, public/private subnets, NAT GW, ALB, Auto Scaling groups, RDS primary → secondary; no SPOF; diagram.
Auto scaling policies; scaling types; launch templates.	Target tracking, simple, step; out/in/up; versioned, recommended templates.
Define RTO/RPO; compare DR strategies.	Definitions; backup & restore, pilot light, warm standby, multi-site with RTO and cost.
Lesson 4 — Consistency, replication, fault tolerance (highest weight in theory)	
Data-centric vs client-centric; describe each model.	Two views; strict, sequential, causal, eventual; four client guarantees with examples; spectrum.
Is this history sequentially/causally consistent?	Find one interleaving for all; check causal (happened-before) pairs.
Push vs pull; replica placement; why replicate.	Read/write ratio, freshness, bandwidth; permanent, server-, client-initiated; performance, reliability, scalability, availability.
CAP in practice; PACELC.	CP (ZooKeeper, quorum DBs), AP (DNS, NoSQL, carts), CA (single site); tuning quorums; Else L vs C.
Failure types; redundancy; fault → error → failure.	Crash, omission, timing, response, Byzantine; information, time, physical redundancy.
Heartbeat vs gossip.	Centralized vs decentralized; speed; scalability; SPOF; best fit; O(log N).
Reliable client-server and group communication.	Four failure points; at-most/at-least/exactly-once; atomic multicast; FIFO/causal/total order.
Explain 2PC and 3PC; why 3PC exists.	Roles, phases, ABORT on any NO, blocking scenario, prepare-to-commit, comparison table, diagram.
Lesson 5 — Cloud infrastructure and virtualization	
Type 1 vs Type 2 hypervisors.	Runs on, performance, installation, use, security, examples; stack diagrams.
Types of virtualization; benefits and limitations.	Server, storage, network, desktop; utilisation, cost, provisioning, DR; overhead, licensing, host SPOF, hyperjacking.
VMs vs containers; when to use each.	Isolation, size, boot, density, use; choice lists; containers inside VMs.
Data centre architecture.	Components; regions and AZs; pooling; multi-tenancy; four layers; Kubernetes; elasticity.
Lesson 6 — Cloud management and Kubernetes	
Resource provisioning and IaC.	Traditional vs cloud; IaC steps, tools, benefits.
Load balancing (types) and auto-scaling (benefits/challenges).	Application/network/global; metrics; horizontal vs vertical.
Kubernetes architecture and objects.	Control plane (API server, scheduler, controller manager, DB) and workers (kubelet, runtime, kube-proxy); Pod, Deployment, Service, Ingress, ConfigMap/Secret, Namespace.
What is a Pod? Lifecycle and restart policies.	Shared IP/volumes, co-scheduled, ephemeral; sidecar and init; phases; Always/OnFailure/Never; Deployment recreates Pods.
Design for resilience / flexibility / cost.	Slide lists 19–21.
Lesson 7 — Storage and DFS	
Block vs file vs object storage.	How it works, latency, scale, metadata, use, examples.
GFS architecture and consistency; GFS vs HDFS.	Master, chunkservers, 64 MB, 3×, metadata vs data path, relaxed consistency; HDFS HA, 128 MB, rack awareness, erasure coding.
Amazon S3; S3 vs EBS vs EFS.	Bucket/object/key, 11 nines, classes, strong consistency, security; comparison table.
DFS design trade-offs.	Consistency vs availability; central vs distributed metadata; replication vs erasure coding; block size.

Lesson 8 — Server virtualization and lab	
Full vs para vs hardware-assisted virtualization.	Binary translation; modified guest/hypercalls; VT-x/AMD-V; examples; performance.
Hardware-assisted CPU, MMU, I/O.	VT-x/AMD-V (XD/NX, power cycle); EPT/RVI vs shadow page tables; VT-d/AMD-Vi, VMDirectPath.
vMotion steps; vMotion vs Storage vMotion.	Nine steps with memory bitmap and quiesce; host vs datastore.
VMware networking; vSwitch vs physical switch.	vNIC, port group, vSwitch, vmnic; no MAC learning, no STP, no trunks, VLAN 802.1Q.
Lab: create a VM and set a static IP.	ESXi steps; nmcli modify with address/24, gateway, DNS, manual; down/up; ip addr, ip route, ping by IP and name.

Part C — Comparison tables to be able to draw

#	Comparison	Rows to include	Lesson
1	Centralized vs parallel vs distributed	Memory, clock, cooperation, failure, example	1
2	Client-server vs P2P vs microservices	Idea, control, scalability, weakness, example	1
3	IaaS vs PaaS vs SaaS	What you get, you manage, examples, best for	2
4	Public vs private vs hybrid vs community vs multi	Definition, pros, cons, example	2
5	Edge vs fog vs cloud	Where, latency, compute, use, examples	2
6	NFV vs SDN	What changes, goal, relationship	3
7	Four DR strategies	Standby, after event, RTO/RPO, cost	3
8	Consistency models	Rule, real time?, cost, example	4
9	Push vs pull	How, freshness, bandwidth, fit	4
10	Heartbeat vs gossip	Architecture, scalability, speed, SPOF, fit	4
11	2PC vs 3PC	Phases, blocking, messages, complexity	4
12	Type 1 vs Type 2	Runs on, performance, install, use, security, examples	5
13	VM vs container	Isolation, size, boot, density, use	5
14	Block vs file vs object; S3 vs EBS vs EFS	Unit, latency, scale, metadata, use; attach point, durability	7
15	GFS vs HDFS	Origin, metadata node, block size, replication, placement, workload	7
16	Full vs para vs hardware-assisted	Mechanism, guest modified?, performance, examples	8

Part D — Exam-style questions with model answers

A mock paper mixing all lessons. Attempt each under timed conditions before reading the answer.

Q1. Define a distributed system. Explain four of its characteristics and two of its challenges. (10 marks)

A distributed system is a collection of independent computers that appears to its users as a single coherent system, communicating over a network to achieve a common goal; nodes do not share memory, cooperate only by messages and have no global clock. Characteristics: resource sharing (hardware, data and services shared, e.g. one file server); scalability (grow by adding nodes horizontally); concurrency (many requests processed simultaneously, so shared data must be protected); transparency (distribution hidden, e.g. location and failure transparency). Challenges: partial failures (some nodes fail while others run, and a node cannot tell crashed from slow); consistency (keeping replicated data in sync under concurrent updates).

Q2. A bank and a social network both replicate data across data centres. Using the CAP theorem, explain which property each should give up during a network partition and why. (8)

CAP: a distributed data store can provide at most two of consistency (every read returns the latest write or an error), availability (every request gets a non-error response) and partition tolerance (keeps operating despite lost/delayed messages). Partitions are unavoidable, so the choice is C or A. The bank should be CP: during a partition a replica that cannot coordinate refuses or delays withdrawals and balance reads, because showing a stale balance could allow overdrawing, a real financial loss. The social network should be AP: replicas keep serving posts that may be slightly stale and converge later (eventual consistency), because an old feed is acceptable but downtime loses users.

Q3. Compare IaaS, PaaS and SaaS using a diagram of the responsibility stack. Give an example and suitable use of each. (10)

(Draw the stack: applications, data, runtime, middleware, OS, virtualization, servers, storage, networking.) IaaS: provider manages virtualization and below; customer manages the OS, middleware, runtime, data and applications; e.g. AWS EC2; for lift-and-shift migrations, dev/test, HPC. PaaS: the provider also manages OS, middleware and runtime; the customer manages only the app and data; e.g. Google App Engine, Heroku; for developers who want to deploy code with auto-scaling, CI/CD and managed databases. SaaS: provider manages everything; customer just uses the software; e.g. Gmail, Microsoft 365; for end users needing ready applications.

Q4. Explain the difference between sequential consistency and eventual consistency. For P1: W(x)a; P2: W(x)b; P3: R(x)a, R(x)b; P4: R(x)a, R(x)b, state whether the history is sequentially consistent. (8)

Sequential consistency: the result is as if all operations were executed in one sequential order seen identically by all processes, respecting each process's program order but not real time. Eventual consistency: if updates stop, all replicas eventually converge to the same value, with no ordering or timing guarantee meanwhile; it favours availability (DNS, NoSQL). The history is sequentially consistent: P3 and P4 both observe a then b, so the single order W(x)a, W(x)b explains every read.

Q5. Describe the Two-Phase Commit protocol with a diagram. Explain the situation in which it blocks and how Three-Phase Commit avoids it. (12)

(Draw coordinator and participants with VOTE-REQUEST, YES/NO, COMMIT/ABORT.) Phase 1: the coordinator asks every participant whether it can commit; each prepares, logs and votes YES or NO. Phase 2: if all voted YES the coordinator broadcasts COMMIT, otherwise ABORT; participants apply it, giving all-or-nothing atomicity. Blocking: if the coordinator crashes after participants voted YES but before they receive the decision, they cannot commit or abort alone and must wait holding locks. 3PC inserts a PREPARE-TO-COMMIT phase acknowledged by participants before the final COMMIT, so every participant knows the group's intent; after a coordinator crash survivors use timeouts or a new coordinator: if any received PREPARE-TO-COMMIT they commit, otherwise they abort. It avoids blocking under crash-only failures without partitions, at the cost of more messages and complexity.

Q6. Compare Type 1 and Type 2 hypervisors, and virtual machines with containers. Recommend a technology for (a) running Windows and Linux on one server, (b) a microservice web shop with sudden traffic spikes. (12)

Type 1 runs directly on hardware (ESXi, Hyper-V, Xen, KVM): near-native performance, smaller attack surface, used in data centres. Type 2 runs as an application on a host OS (VirtualBox, VMware Workstation): easy to install, more overhead, inherits host OS risks, used on laptops. VMs include a full guest OS: strong isolation, GBs, minutes to boot, lower density. Containers share the host kernel: weaker isolation, MBs, seconds to start, higher density. (a) VMs on a Type 1 hypervisor, since different guest operating systems are needed. (b) Containers orchestrated by Kubernetes (possibly inside VMs for tenant isolation), since microservices need fast startup, density and rapid horizontal scaling.

Q7. Draw a highly available three-tier web architecture on AWS and explain how it survives the loss of an availability zone. Define RTO and RPO and recommend a DR strategy for an online shop that may be down for at most 30 minutes. (12)

(Draw VPC across two AZs: public subnets with NAT gateways, private subnets with web and app servers in Auto Scaling groups, an ALB in front, RDS primary in AZ1 replicating to a secondary in AZ2.) If AZ1 fails, the ALB's health checks send traffic only to AZ2, the Auto Scaling group launches replacement instances there, and RDS fails over to the secondary, so no single point of failure stops the service. RTO is the maximum acceptable downtime; RPO is the maximum acceptable data loss measured in time. With an RTO of about 30 minutes, pilot light suits: the database is continuously replicated to the DR region (small RPO) and application servers are started and DNS (Route 53) switched over in tens of minutes, more cheaply than warm standby or multi-site.

Q8. Explain the Kubernetes architecture and the role of a Pod. What happens when (a) a container in a Pod crashes, (b) the node running a Deployment's Pod fails? (10)

A cluster has a control plane (API server as the entry point, scheduler choosing nodes, controller manager keeping actual state equal to desired state, cluster database storing state) and worker nodes (kubelet running and reporting Pods, container runtime, kube-proxy for Service networking). A Pod is the smallest deployable unit: one or more containers that share one IP (localhost), optional volumes, are co-scheduled on one node and are ephemeral. (a) The kubelet restarts the container in place according to the restart policy (Always by default), with growing backoff (CrashLoopBackOff). (b) The Pod is lost; the Deployment controller sees fewer replicas than desired and the scheduler places a new Pod on a healthy node.

Q9. Explain the GFS architecture with a diagram. Why does data not flow through the Master? Compare GFS with HDFS. (12)

(Draw client, Master, chunkservers with replicated chunks.) A single Master keeps metadata (namespace, file-to-chunk mapping, chunk locations); chunkservers store 64 MB chunks, each replicated 3× across machines/racks. A client asks the Master for the chunk handle and locations, then reads/writes directly with chunkservers. Separating the metadata path from the data path stops the single Master becoming a throughput bottleneck. HDFS: open source (Hadoop); NameNode/DataNodes; 128 MB blocks; Active/Standby NameNodes for HA; explicit rack awareness; erasure coding in 3.x; general Hadoop workloads. GFS: internal Google, single Master, 64 MB, basic placement, Google batch workloads.

Q10. Compare full virtualization, para-virtualization and hardware-assisted virtualization. Explain vMotion. (10)

Full virtualization: the hypervisor uses binary translation to rewrite sensitive instructions, so unmodified guests run (ESXi, Hyper-V). Para-virtualization: the guest kernel is modified to make hypercalls, giving the best performance but needing OS changes (Xen, Oracle VM). Hardware-assisted: CPU extensions (Intel VT-x, AMD-V) trap sensitive instructions in hardware, so unmodified guests run with simpler, faster hypervisors; MMU (EPT/RVI) and I/O (VT-d/AMD-Vi) extensions extend this. vMotion live-migrates a running VM between ESX hosts without downtime: vCenter checks compatibility, registers the VM on the target, copies memory and state while tracking changed pages in a bitmap, briefly quiesces the VM, copies the bitmap, starts it on the target, makes a final memory copy and unregisters it from the source; shared VMFS storage means disks do not move.

Common mistakes in exam answers

Mistake	Fix
Writing a list of words without explanation	One explained point per mark, with an example.
"Compare" answered as two separate paragraphs	Use a table with the same rows for both sides.
No diagram for an architecture question	Draw and label it; it earns marks and saves words.
Confusing service and deployment models	IaaS/PaaS/SaaS = what; public/private/hybrid = who owns/uses.
Saying CA is a good choice for big distributed systems	Partitions happen; choose C or A.
2PC "blocks when a participant says NO"	It aborts; it blocks when the coordinator fails after YES votes.
Mixing RTO and RPO; vMotion and Storage vMotion; image and container; edge and fog	Learn the Part A definitions word for word.

Cheat sheet

Must-know lists	
DS characteristics	Resource sharing, scalability, concurrency, fault tolerance, no global clock, transparency
DS challenges	Complexity, partial failures, consistency, synchronization, security, network reliability
NIST 5	On-demand self-service, broad network access, resource pooling, rapid elasticity, measured service
5 V's	Volume, velocity, variety, veracity, value
Zero Trust	Verify explicitly, least privilege, assume breach
Failures	Crash, omission, timing, response, Byzantine
Redundancy	Information, time, physical
Client-centric	Monotonic reads, monotonic writes, read your writes, writes follow reads
Virtualization types	Server, storage, network, desktop
HV techniques	Full (binary translation), para (hypercalls), hardware-assisted (VT-x/AMD-V)
K8s objects	Pod, Deployment, Service, Ingress, ConfigMap/Secret, Namespace
LB types	Application, network, global
Scaling policies	Target tracking, simple, step
DR	Backup & restore, pilot light, warm standby, multi-site
DFS trade-offs	C vs A, metadata, replication vs EC, block size

Must-know numbers	
Exam	2 h, closed book, 50%
Edge/fog/cloud latency	<10 / 10–50 / 50–150 ms
Market share 2024	AWS 31, Azure 25, GCP 11, Alibaba 4 (%)
Savings	Reserved up to 72%; spot 60–90%
Pre-virtualization CPU	10–15%
AWS AZs	≥ 2 per region
GFS / HDFS block	64 MB / 128 MB; 3× replicas; EC ~1.5×
S3	11 nines durability; strong consistency since Dec 2020; Deep Archive up to 12 h
EBS	gp3 3,000 IOPS; io2 256,000 IOPS
EPT / RVI gains	Up to 48% / 42%
Gossip	O(log N) rounds
Quorum	R + W > N
Lab IP	192.168.8.X/24, gw 192.168.8.1, DNS 8.8.8.8 1.1.1.1

Must-draw diagrams
DS single view · client-server / P2P · CAP triangle · IaaS/PaaS/SaaS stack · VM vs container stack · Type 1 vs Type 2 · Docker architecture · AWS HA 3-tier · DR spectrum · 2PC time line · heartbeat · K8s cluster · Pod anatomy · GFS · HDFS racks · vSwitch layout